

Finite state machine with VHDL

Lecture 7



**Electronic and Communication Engineering
Department**

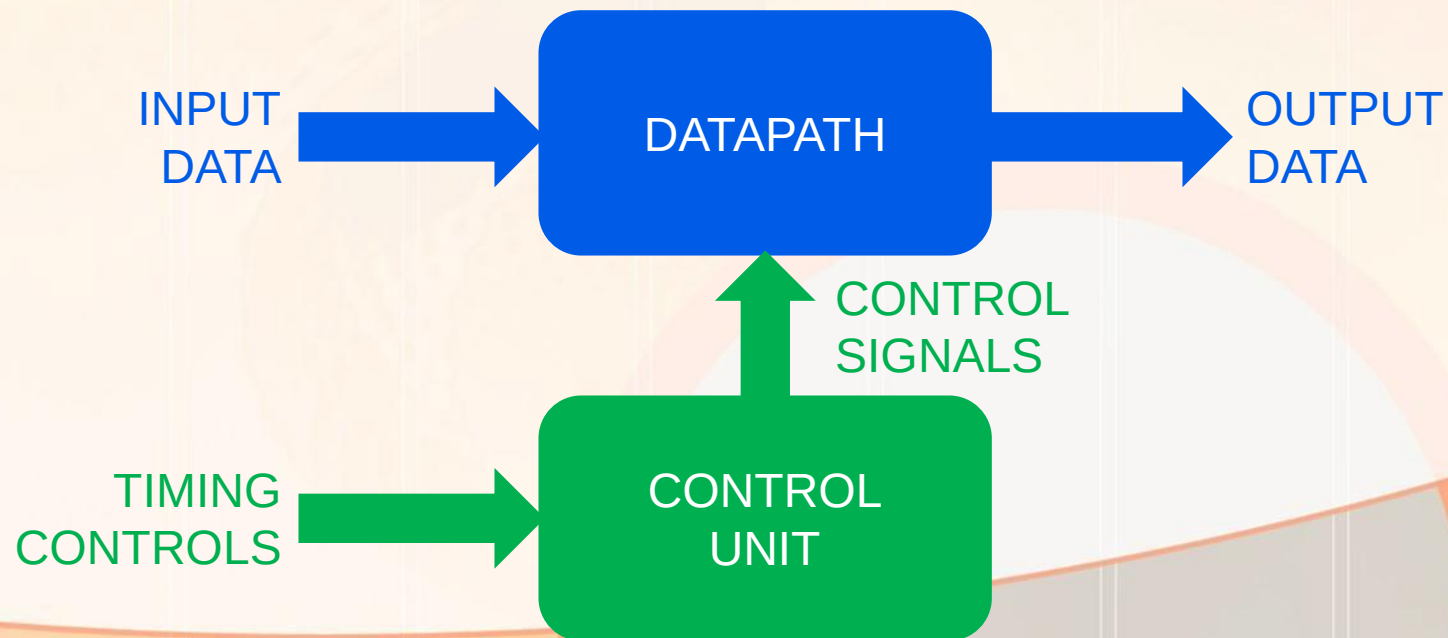
ECE 307 Introductory to Digital Systems Lab

Dr. Ammar Jallawi Mahmood

Ammar.karkar@uokufa.edu.iq

Datapath & control

- *Many (most?) complex circuits consist of:*
 - a **datapath**, which implements the **algorithm** to be realized by the circuit on the data
 - a **control unit**, which implements the **timing** of the algorithms and directs the operation of the datapath



Datapath & control

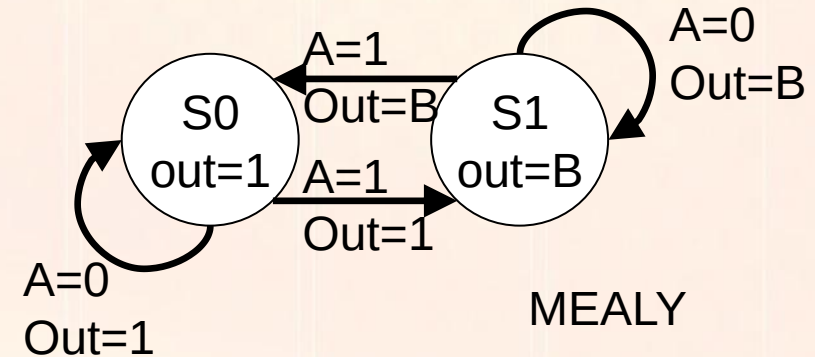
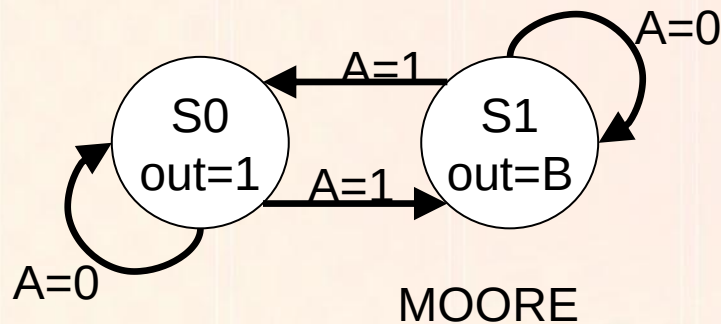
- *Many (most?) complex circuits consist of:*
 - *a **datapath**, which implements the **algorithm** to be realized by the circuit on the data*
 - *a **control unit**, which implements the **timing** of the algorithms and directs the operation of the datapath*
- *Datapaths usually consist of **operations** applied to data, represented by **arrays/vectors***
 - *Understanding **libraries** and **array operations** is fundamental to realize efficient datapaths*
- *Control units usually consist of complex **sequence control mechanisms**, often implemented using **Finite State Machines (FSM)***
 - *Understanding **FSM implementation methods** is fundamental to realize efficient control units*

Finite State Machines - Design

- *Finite State Machines (FSMs) are a fundamental component of complex logic design*
 - *For example, FSMs are used to **control the sequential operation of a circuit** (for example, to direct a data path or to interface with devices that have a timing component)*
- *The **design** of a FSM usually starts by a set of **specifications**, which can be in many forms*
 - ***Textual descriptions** are common, but most often the state machine behaviour is determined **implicitly** by the system it has to control (e.g. multipliers)*
- *From the specifications, the designer has to determine a set of **states, inputs, and outputs**, and a set of **transition rules** that control how the machine moves from one state to another.*
- *This process is fundamentally a matter of experience, as there are only few, very limited automated methodologies*

Finite State Machines - Design

- Normally, the operation of a FSM is described using a *state graph*.



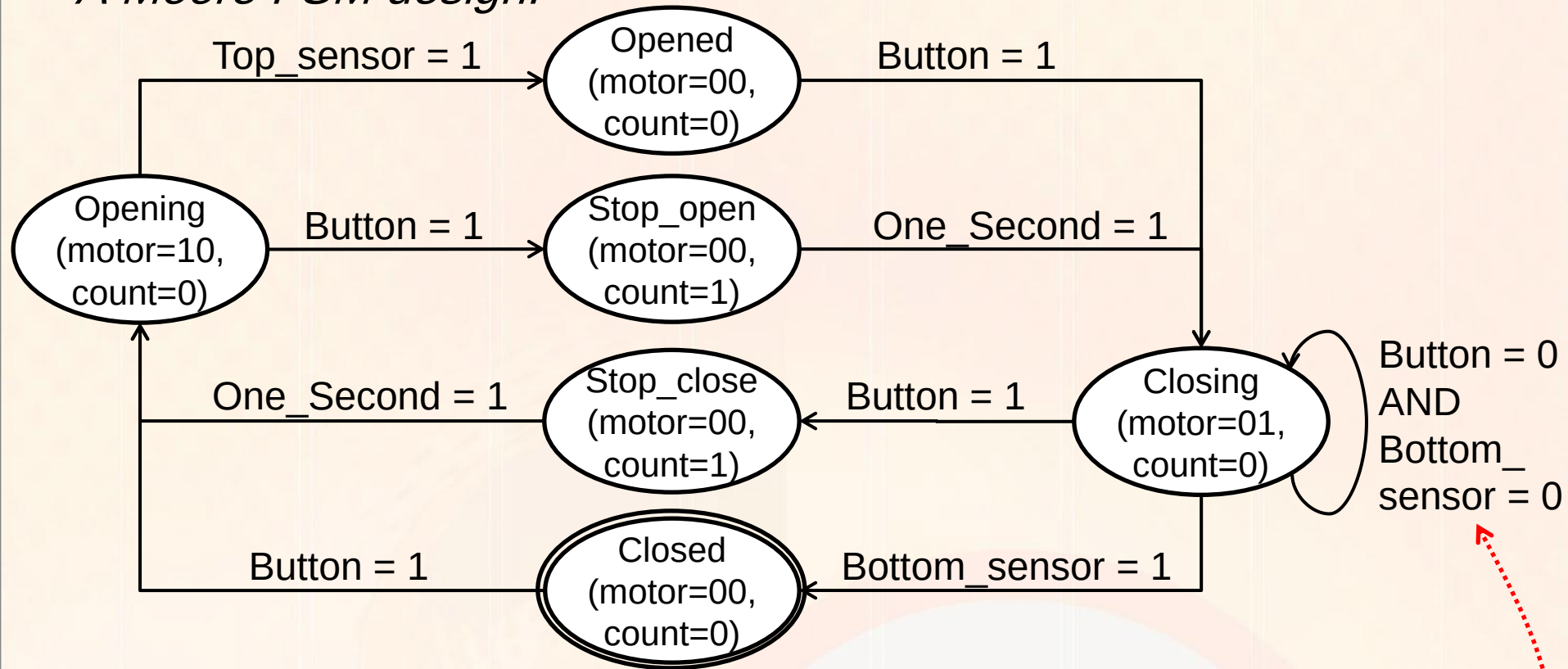
- The *inputs* to the FSM usually determine state transitions or outputs
- The *outputs* of an FSM can depend only on the current state of the machine (*Moore FSM*) or on a combination of states and inputs (*Mealy FSM*).
 - The choice of one over the other depends strongly on the specific needs of the application
- Optimisation of the state graph is important, but not vital in most cases (and automated tools do exist!)

Finite State Machines – Design example

- *As an example, suppose we want to design a FSM to control a **garage door motor***
- *The (textual) specifications are as follows:*
 - *The operation is controlled by a single button on a remote control*
 - *If the door is closed, the button starts to open the door*
 - *If the door is open, the button starts to close the door*
 - *When a top sensor detects that the door is fully open, the motor should switch off*
 - *If the user presses the button when the door is closing, the door stops for one second, then re-opens. The delay is implemented by a counter.*
- *The door motor controls are encoded as follows:*
 - *motor off = 00*
 - *motor up (opens the door) = 10*
 - *motor down (closes the door) = 01*

Finite State Machines – Design example

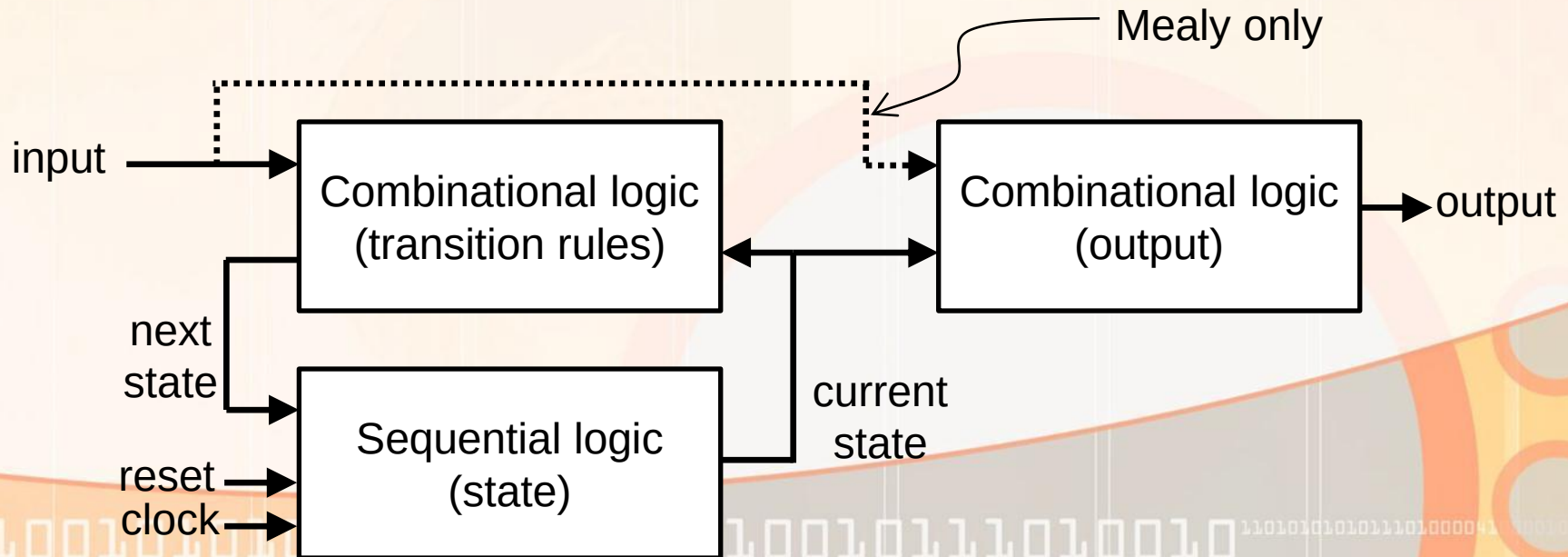
- A Moore FSM design:



- In this case, the FSM will have **four inputs** (Button, Top_Sensor, Bottom_Sensor, One_Second), **two outputs** (motor, count), and **six states**
- Note that a **reset state** must always be identified (double circle)
- Also note that, to be fully complete, **same-state looping arrows** should be included in the state graph for all states

Finite State Machines

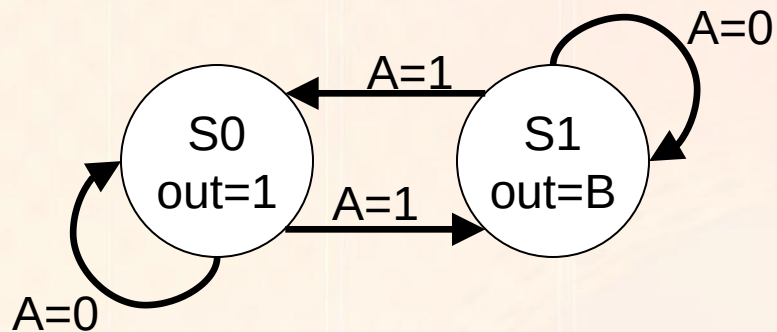
- *The hardware implementation of a finite state machine implies the design of **three sub-systems**:*
 - *A sequential logic subsystem (1 or more flip-flops) to store the **current state***
 - *A combinational logic subsystem to compute the **next state** as a function of the current state and of the inputs*
 - *A combinational logic subsystem to compute the **output** as a function of the current state (Moore FSMs) and possibly of the inputs as well (Mealy FSMs)*



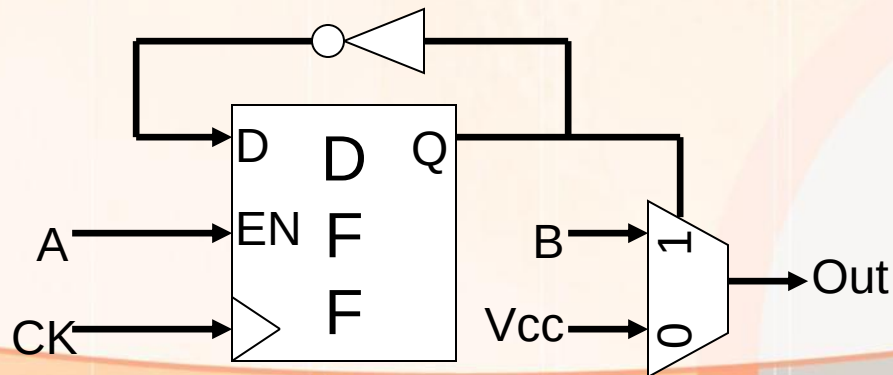
Finite State Machines

Implementing all three subsystems as a single process in VHDL is very “dangerous”

- *Very very simple Moore FSM:*



- *As a circuit, it might look like:*

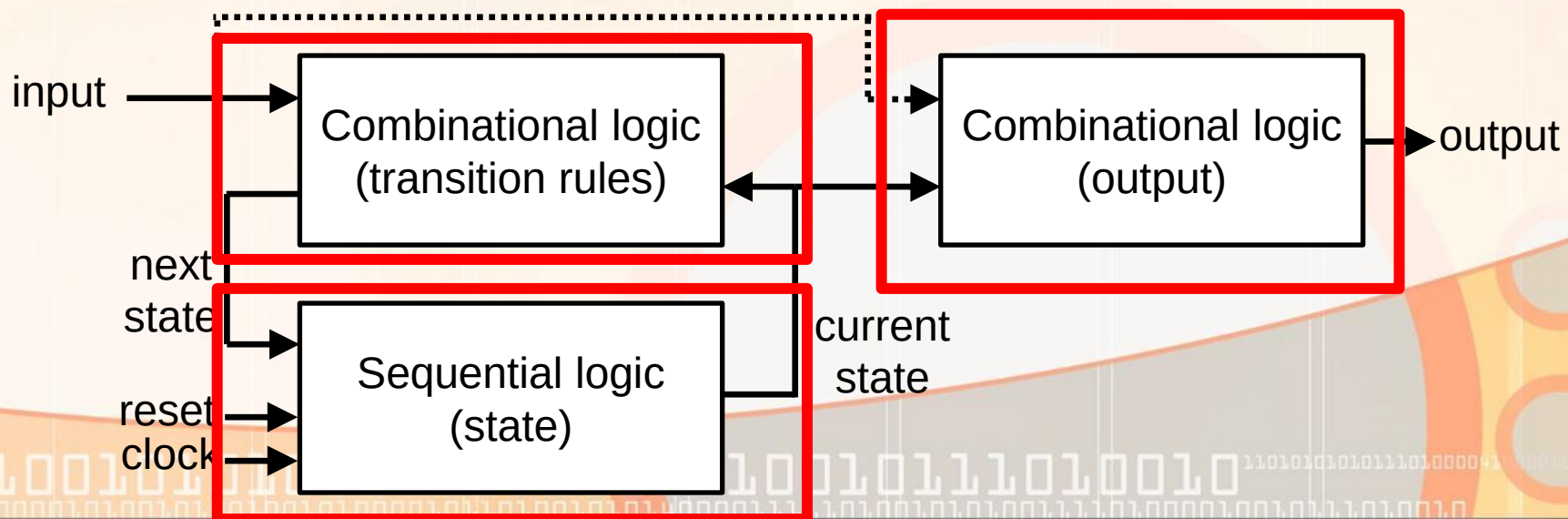


```
[...]
type fsm_states is (s0, s1);
signal state: fsm_states;
[...]
fsm: process (clock) is
begin
    if rising_edge(clock) then
        if (reset = '1') then
            state <= s0;
        else
            case state is
                when s0 =>
                    output <= '1';
                    if a = '1' then
                        state <= s1;
                    end if;
                when s1 =>
                    output <= b;
                    if (a = '1') then
                        state <= s0;
                    end if;
            end case;
        end if;
    end if;
end process fsm;
```

BAD

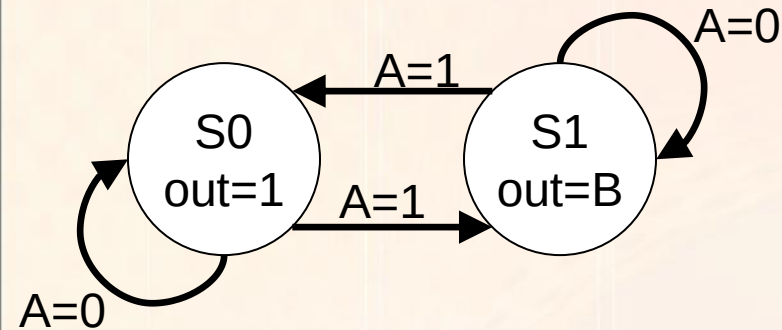
Finite State Machines

- *Implementing a FSM as a single process is risky because it is a complex mix of sequential and combinational logic, the probability of **errors** (particularly in synthesis) is high.*
- *To reduce the risk of errors, it is usually a good idea to adopt a **VHDL implementation methodology** that recognizes the different components of a FSM.*
- *In particular, in this module we will use a methodology that **separates each part of the FSM***



Finite State Machines

- *The implementation methodology is the following:*
 - *First, define an entity for the state machine*
 - *It is usually a good idea to have a separate entity for FSMs (or at least for the control logic)*



```
entity good_fsm is
    port (rst,clk,a,b: in std_logic;
          output : out std_logic);
end good_fsm;
```

- *Then, **declare the states as an enumerated type** and two signals (state, next_state) of that type.*

```
architecture good of good_fsm is
    type fsm_states is (s0, s1);
    signal state, next_state: fsm_states;
```

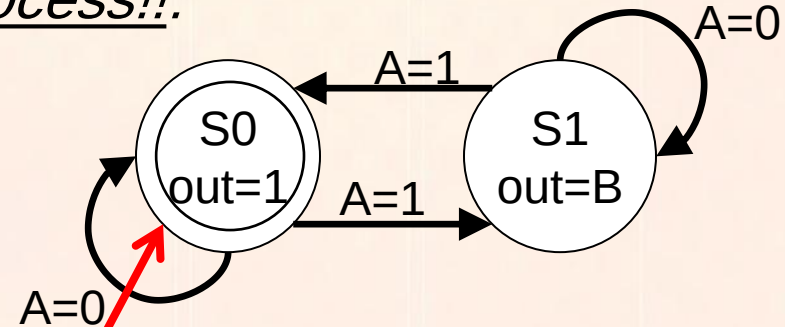
*Keeping in mind that it's always a good idea to **assign meaningful names to the states** (in this abstract example, obviously it does not matter)*

Finite State Machines

- Then, *define the state register* (and only the state register) *with a D-type register with synchronous reset* - only the next state -> state assignment should be included in this process!!:

**Synchronous process
(sensitive only to clock)**

```
state_assignment: process (clk) is
begin
    if rising_edge(clk) then
        if (reset = '1') then
            state <= s0;
        else
            state <= next_state;
        end if;
    end if;
end process state_assignment;
```

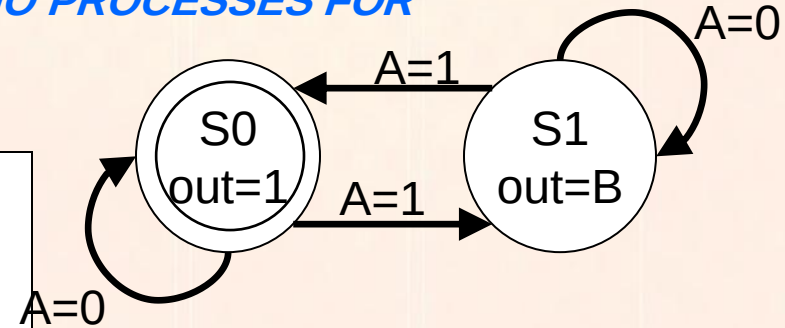


**Define a reset state and
use a synchronous reset at
the start of the process**

Finite State Machines

- Then, *define the transition rules* (and only the transition rules) *with a case statement in an asynchronous process* –
- *THIS IS THE ONLY EXCEPTION TO THE RULE OF "NO PROCESSES FOR COMBINATIONAL LOGIC"*

```
transitions: process (state, ina) is
begin
  case state is
    when s0 =>
      if ina = '1' then
        next_state <= s1;
      else
        next_state <= state;
      end if;
    when s1 =>
      if (ina = '1') then
        next_state <= s0;
      else
        next_state <= state;
      end if;
  end case;
end process transitions;
```



Asynchronous process: The sensitivity list includes the current state and all inputs to the FSM. Only the next state should be assigned inside this process!!

For each case (state), use "if" statements for all outgoing arrows in the state graph

"Looping" arrows (conditions when the state has to remain the same) must always be specified

Finite State Machines

- Finally, *assign all outputs* using combinational logic outside of the process:

```
output <= b when state = s1 else '1';
```

The output can always be computed with combinational logic as a function of the state (Moore FSM) or of the state plus the inputs (Mealy FSM). The “when-else” statement is particularly useful.

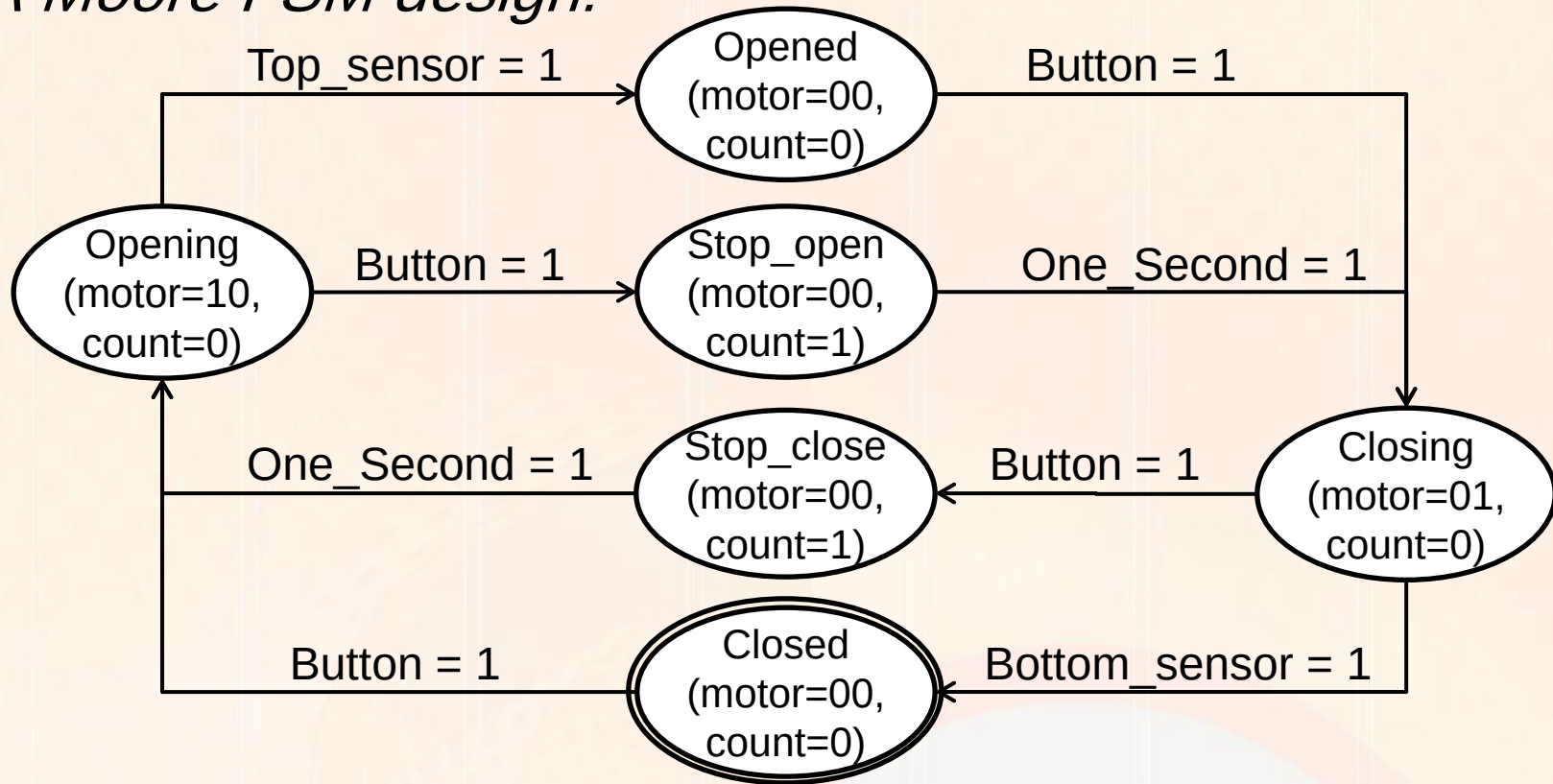
- *There are other, equally good methodologies – for example, you could have a single process that generates the value of the next state and assigns it. But in general you should always follow a design methodology for FSMs.*

Finite State Machines – Design example

- *Garage Door*
- *States (Closed, Open, Opening, Closing, Stopped open, Stopped close)*
- *The FSM will have **four inputs** (button, top_sensor, bottom_sensor, one_second), **two outputs** (motor, count), and **six states***
- *Motor (00: stopped, 10: opening, 01: closing)*

Finite State Machines – Design example

- A Moore FSM design:*



- Note that a **reset state** must always be identified (double circle)*
- Also note that, to be fully complete, **same-state arrows** should be included in the state graph*

Finite State Machines – Design example

- *First, define a new entity and the internal states as an enumerated type:*

```
entity garage_door_FSM is
    port (reset, clk : in std_logic;
          button, bottom_sensor, top_sensor,
          one_second : in std_logic;
          motor : out std_logic_vector (1 downto 0);
          count : out std_logic );
end garage_door_FSM ;

architecture arch of garage_door_FSM is
    type state_type is (opened, closed, opening, closing,
                        stop_open, stop_close);
    signal state, next_state: state_type;

begin
```

Finite State Machines – Design example

- *Next, define a process for the state register only:*

```
state_assignment: process (clk) is
begin
    if rising_edge(clk) then
        if (reset = '1') then
            state <= closed;
        else
            state <= next_state;
        end if;
    end if;
end process state_assignment;
```



Reset state (see
state graph)

Finite State Machines – Design example

- *Next, define a process for the state transitions only:*

```
fsm_process: process (state, button, bottom_sensor,  
                    top_sensor, one_second) is  
begin  
  case state is  
    when closed =>  
      if button='1' then next_state<=opening;  
      else next_state <= state; end if;  
    when opening =>  
      if button='1' then next_state<=stop_open;  
      elsif top_sensor='1' then next_state<=opened;  
      else next_state <= state; end if;  
    when opened =>  
      if button='1' then next_state<=closing;  
      else next_state <= state; end if;  
    when closing =>  
      if button='1' then next_state<=stop_close;  
      elsif bottom_sensor='1' then next_state<=closed;  
      else next_state <= state; end if;  
    when stop_open =>  
      if one_second='1' then next_state<=closing;  
      else next_state <= state; end if;  
    when stop_close =>  
      if one_second='1' then next_state<=opening;  
      else next_state <= state; end if;  
  end case; end process fsm_process;
```



state and ALL inputs that
can change the state

ONLY
next_state
assignments

In your code,
don't write these
in a single line!

Finite State Machines – Design example

- *Finally, define the outputs:*

```
count <= '1' when state = stop_open or  
        state = stop_close else  
        '0';  
  
motor <= "01" when state = closing else  
        "10" when state = opening else  
        "00";  
  
end arch;
```

- *Note that in both examples the outputs were quite simple. This is actually often the case, but they can be combinational logic expressions of any complexity, representing any combinations of states and inputs.*

Control logic

- *Your control logic might include other elements in addition to a FSM!*
 - *Typical examples are combinational operators (adders, etc.), registers and counters (for example, to wait for a number of clock cycles or to generate memory addresses)*
- *If that is the case, it is always much safer to **separate these elements from the FSM and implement them outside of the FSM process** (note that this is a requirement in this module)*
 - *The interactions between these elements and the FSM should be implemented with **dedicated control signals***

Control logic - example

- Example: adding a one-second delay (100MHz clock)*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity control_logic is
    port (reset, clock : in std_logic;
          button, bottom_sensor,
          top_sensor: in std_logic;
          motor : out
          std_logic_vector(1 downto 0));
end control_logic;

architecture arch of control_logic is
    component garage_door_fsm
        port(reset : in std_logic;
             clock : in std_logic;
             button : in std_logic;
             bottom_sensor : in std_logic;
             top_sensor : in std_logic;
             one_second : in std_logic;
             motor : out
             std_logic_vector(1 downto 0);
             count : out std_logic);
    end component;

    signal one_second : STD_LOGIC;
    signal count_en : STD_LOGIC;
    signal counter : UNSIGNED (26 downto 0);
```

```
begin
    door: garage_door_fsm port map(
        reset => reset,
        clock => clock,
        button => button,
        bottom_sensor => bottom_sensor,
        top_sensor => top_sensor,
        one_second => one_second,
        motor => motor,
        count => count_en);

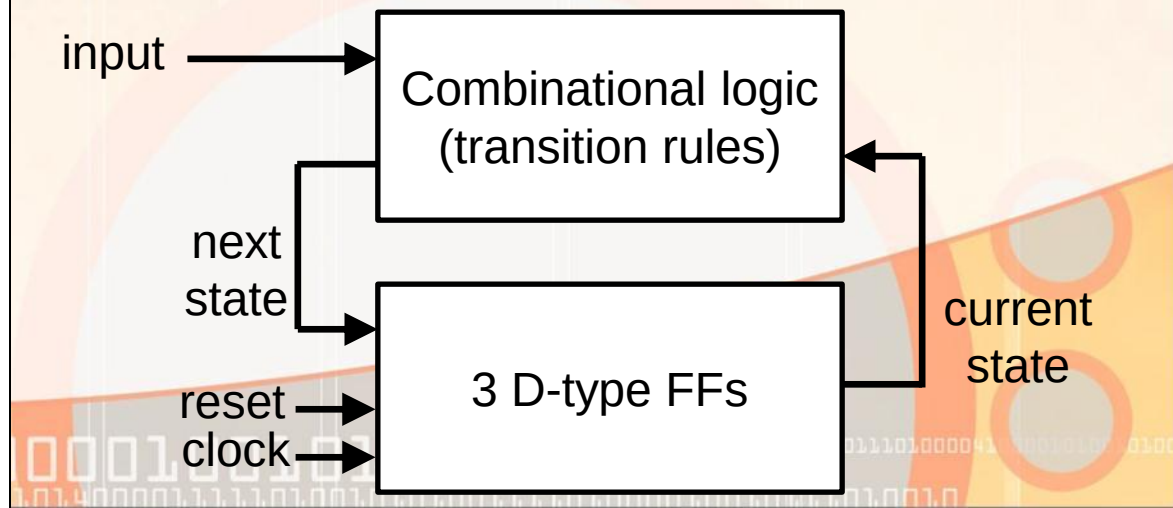
    count_one_sec : process (clock)
    begin
        if (clock'event and clock='1') then
            if (reset = '1') then
                counter <= (others => '0');
            else
                if (count_en = '1') then
                    if (counter = 100000000) then
                        counter <= (others => '0');
                    else
                        counter <= (counter + 1);
                    end if; end if; end if; end if;
                end process count_one_sec;
            one_second <= '1' when (counter=100000000
                                   and count_en = '1') else '0';
        end arch;
```

Finite State Machines – Hardware

- *Once the FSM has been correctly implemented in VHDL, the synthesis tools will generate the hardware required: FFs to store the state, combinational logic to compute the next state.*
- *FSM have several **hardware implementation methodologies***
 - *The one you are probably familiar with is the **minimal** implementation – the minimal number of flip-flops is used to encode the states*

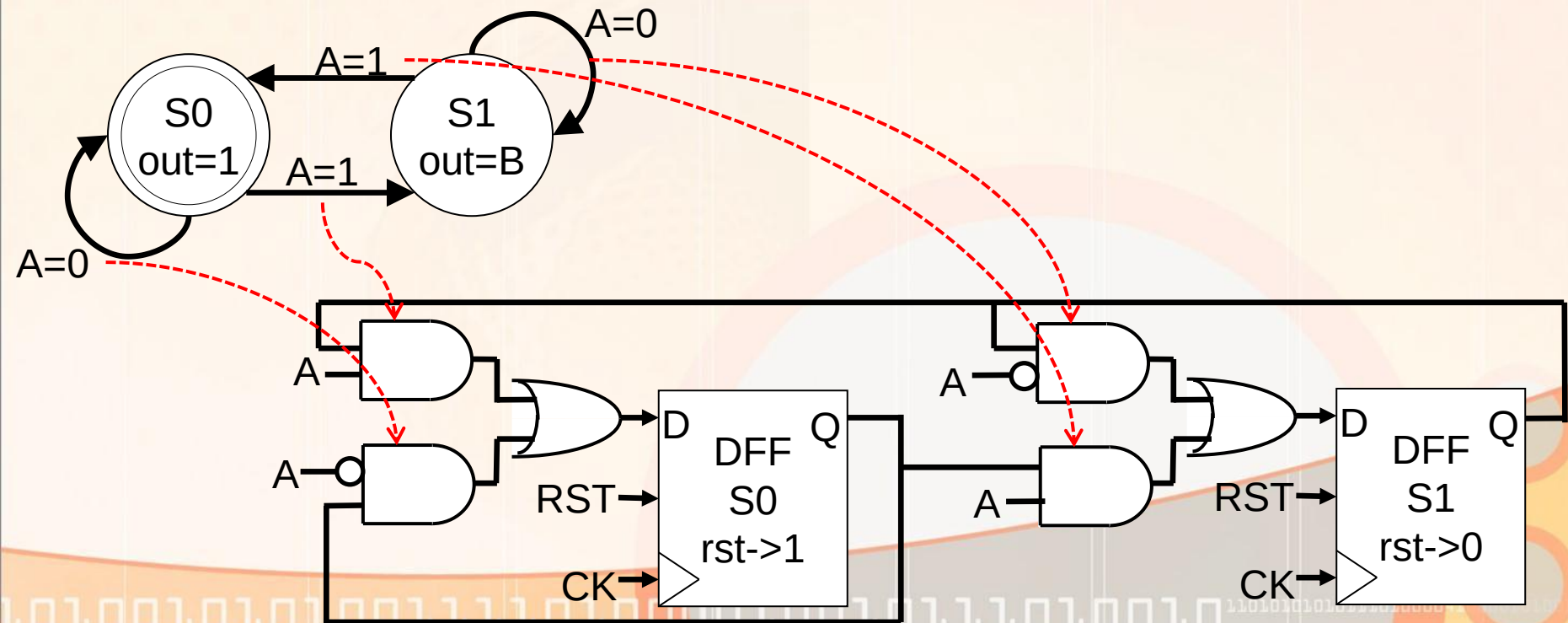
```
type state_type is (opened, closed, opening, closing,  
                    stop_open, stop_close);
```

State	Encoding
opened	001
closed	000
opening	011
closing	010
stop_open	100
stop_close	101



Finite State Machines – Hardware

- *FSM have several **hardware encoding strategies***
 - *But probably the most commonly used in hardware design is **one-hot encoding***
 - *In this encoding, one FF is used for each state*
 - *All FFs are 0 except the one that corresponds to the current state, which holds value 1 (the FF corresponding to the reset state must be reset to 1 and not to 0)*



Finite State Machines – Hardware

- *For the garage door one-hot encoding*

```
type state_type is  
    (opened, closed,  
     opening, closing,  
     stop_open, stop_close);
```

State	Encoding

opened	000010
closed	000001
opening	001000
closing	000100
stop_open	010000
stop_close	100000

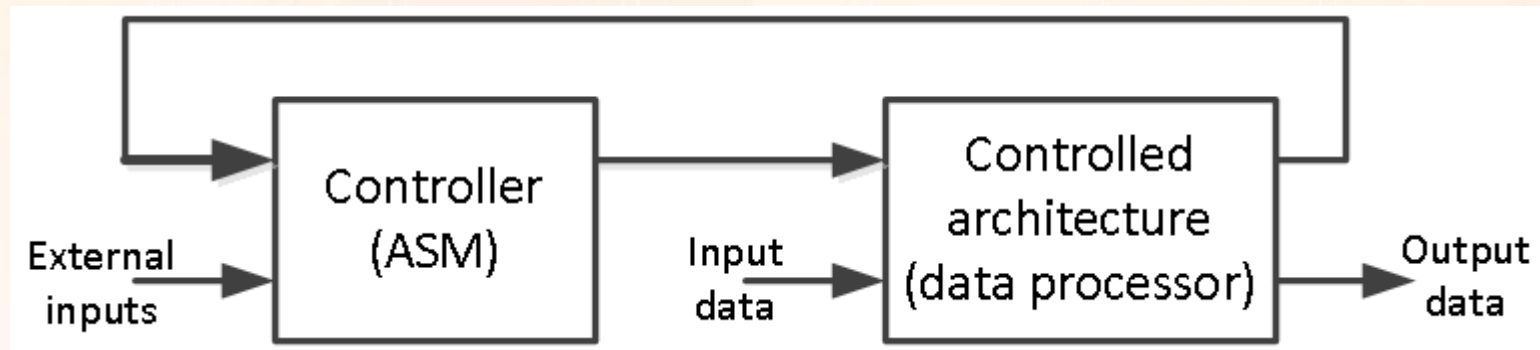
Finite State Machines – Hardware

- To specify the state coding we use the attribute command:*

```
attribute syn_encoding : string;  
attribute syn_encoding of State_type : type is "000010  
000001 001000 000100 100000 010000 ";
```

Algorithmic State Machine (ASM)

- Just as flowcharts are useful in software design, special flowcharts called **Algorithmic State Machines (ASM)** are useful in digital systems hardware design.
- As the control path (behavior) of the system becomes complex, it becomes increasingly difficult to design the control path using the state graph technique.
- The ASM charts technique becomes useful and handy in designing complex and algorithmic circuits.
- The following diagram shows a complex digital system, partitioned into a controller (to generate the control signals) and the controlled architecture (data processor).

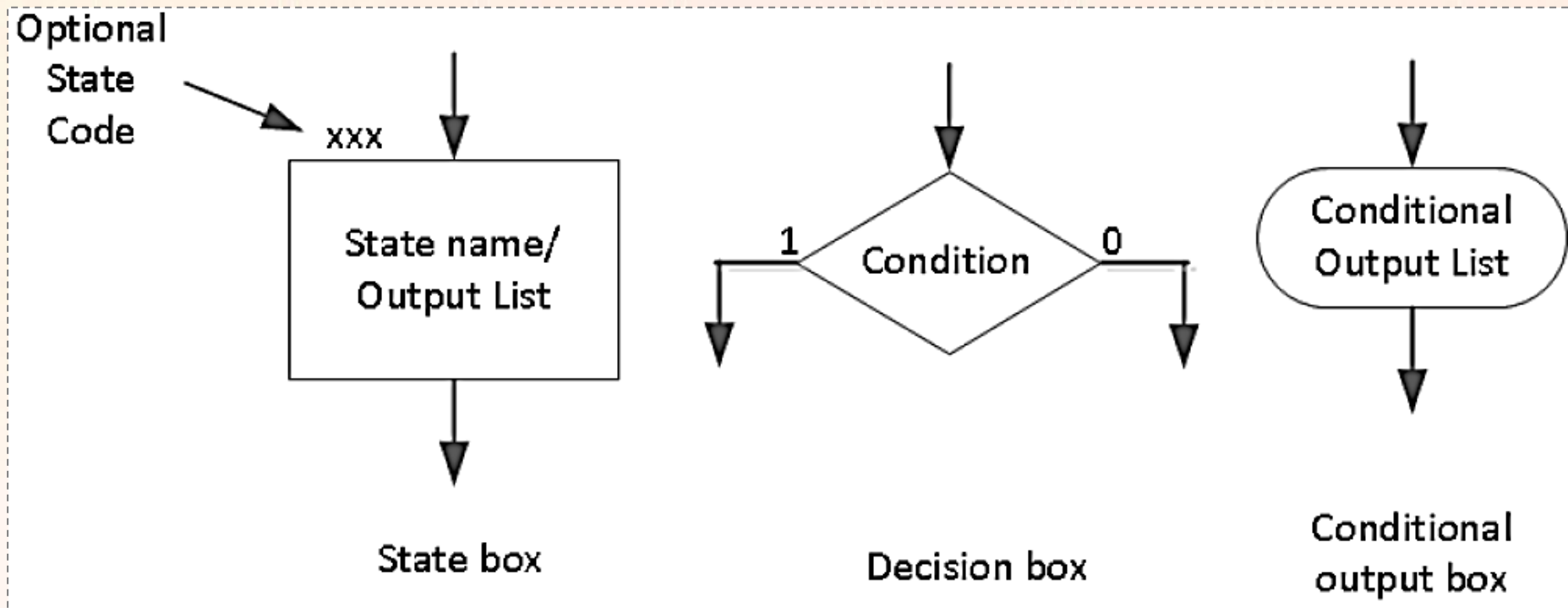


Algorithmic State Machine (ASM)

- The ASM chart differs from an ordinary flowchart in that specific rules must be followed in constructing the chart. When these rules are followed, the ASM chart is equivalent to a state graph, and it leads directly to a hardware realization.

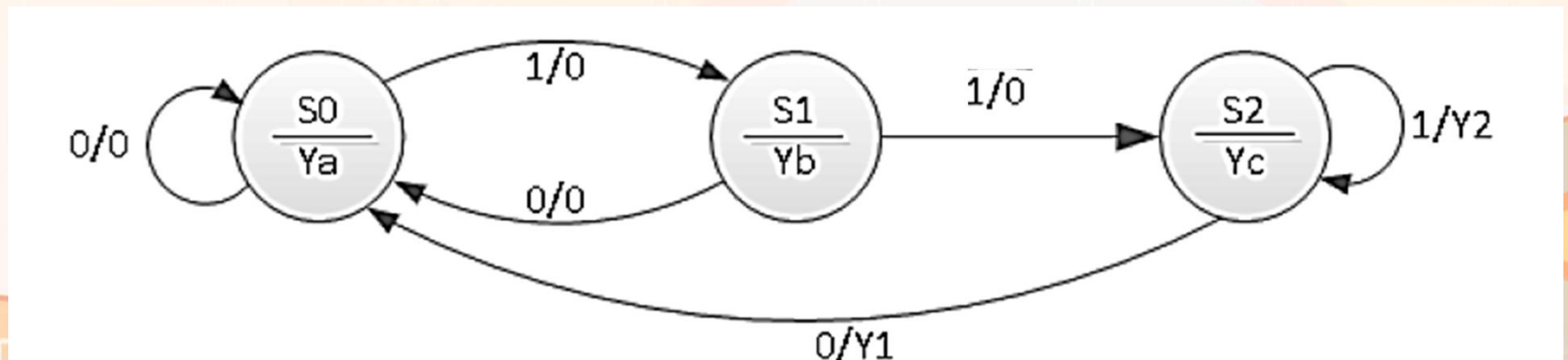
Algorithmic State Machine (ASM)

- The following diagram shows the three main components of an **ASM chart**.

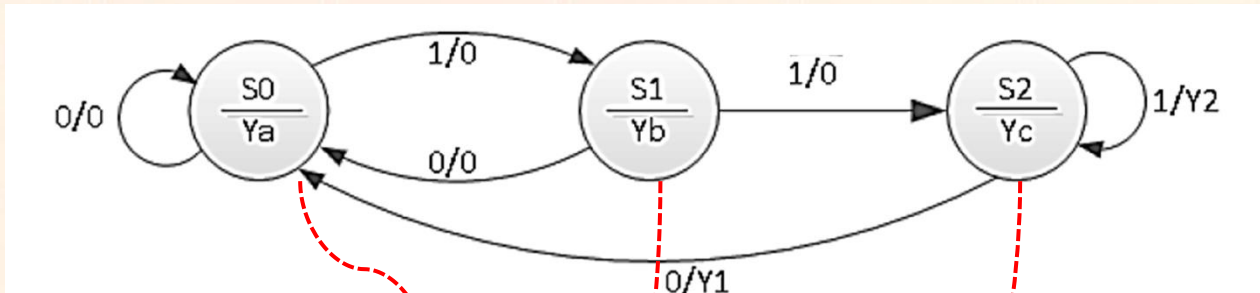


State Machine to ASM Example

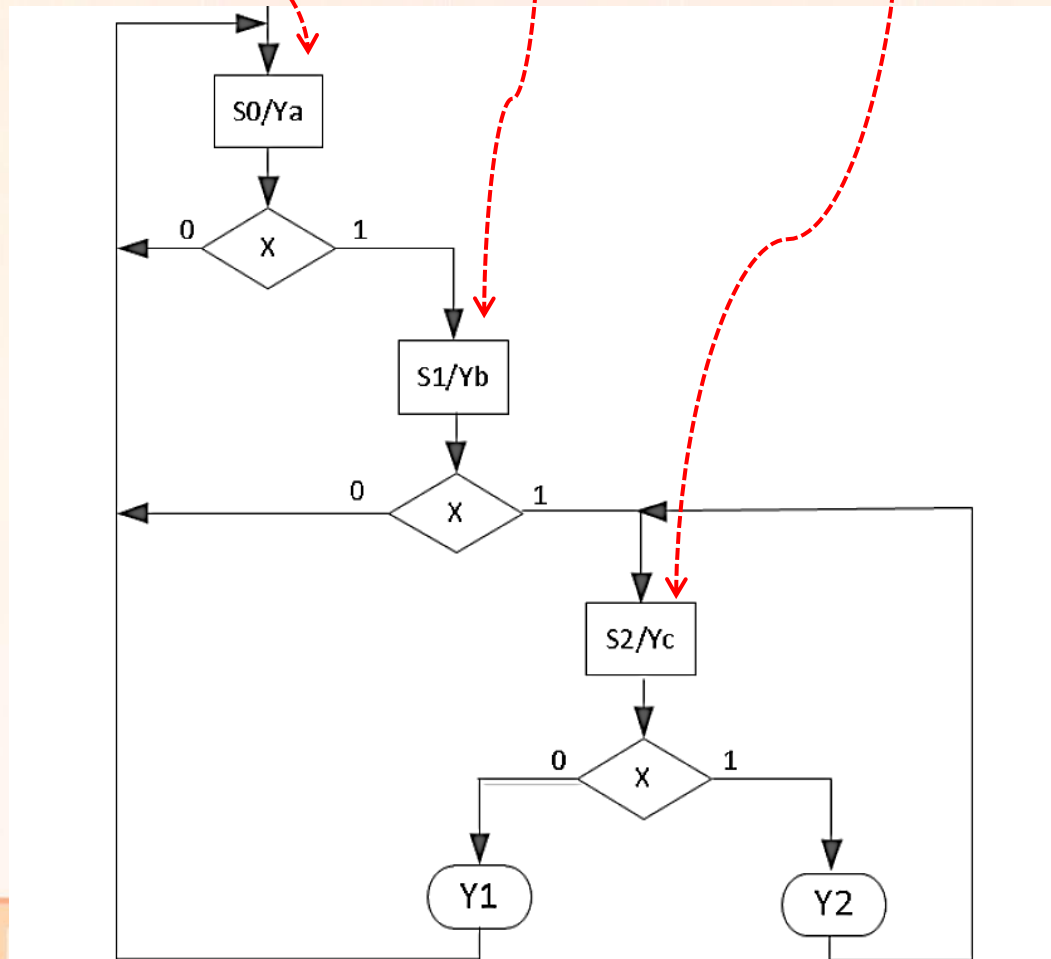
- The state graph below has both Mealy and Moore outputs.
 - Y1 and Y2 are the Mealy outputs and so should be conditional outputs.
 - Ya, Yb, and Yc are the Moore outputs so they should be part of state box. Input
- X can either be “0” or “1” and hence it should be part of the decision box.



State Machine to ASM Example



State Machine



ASM

ASM Chart to VHDL

- Once the **ASM chart** is determined, the conversion to *HDL* is straight forward.
- A **case statement** can be used to specify what happens in each state.
- Each condition box corresponds directly to **an *if statement*** (or an **else if**).
- The following code represents the functionality of the above ASM chart.

ASM to VHDL

```
type state_type is (S0, S1, S2);
signal state, next_state : state_type;
SYNC_PROC : process (clk)
begin
    if rising_edge(clk) then
        state <= next_state;
    end if;
end process;

NEXT_STATE_DECODE : process (state, x) --
    Process to compute both Mealy output &
    nextstate
begin
    y1 <= '0';
    y2 <= '0';

    case (state) is
        when S0 =>
            if (x = '1') then
                next_state <= S1;
            else
                next_state <= S0;
            end if;
        when S1 =>
            if (x = '1') then
                next_state <= S2;
            else
                next_state <= S0;
            end if;
    end case;
end process;
```

```
when S2 =>
    if (x = '1') then
        y2 <= '1';
        next_state <= S2;
    else
        y1 <= '1';
        next_state <= S0;
    end if;

when others =>
    next_state <= S0;

end case;
end process;

OUTPUT_DECODE : process (state) -- Process to
    compute Moore output
begin
    case (state) is
        when S0 => ya <= '1';
        when S1 => yb <= '1';
        when S2 => yc <= '1';
        when others =>
            ya <= '0';
            yb <= '0';
            yc <= '0';
        end case;
    end process;
```


Thank you!

